virtua

Vesting

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

27 November 2023

Introduction **3**

About Virtua 3

About ImmuneBytes 3

Documentation Details **4**

Audit Goals **4**

Audit Process & Methodology **5**

Audit Details **6**

Security Level References **7**

Severity Status References **8**

Severity Status **8**

Finding **9**

Critical Severity Issues **10**

High Severity Issues **10**

Medium severity issues **12**

Low severity issues **13**

Informational **15**

Concluding Remarks **16**

Disclaimer **16**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Virtua

Virtua is a sustainable metaverse platform that provides immersive social, gaming, digital collectible and creative experiences, through NFTs and blockchain technology.

Visit https://virtua.com/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. VANARY Vesting

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors which included -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

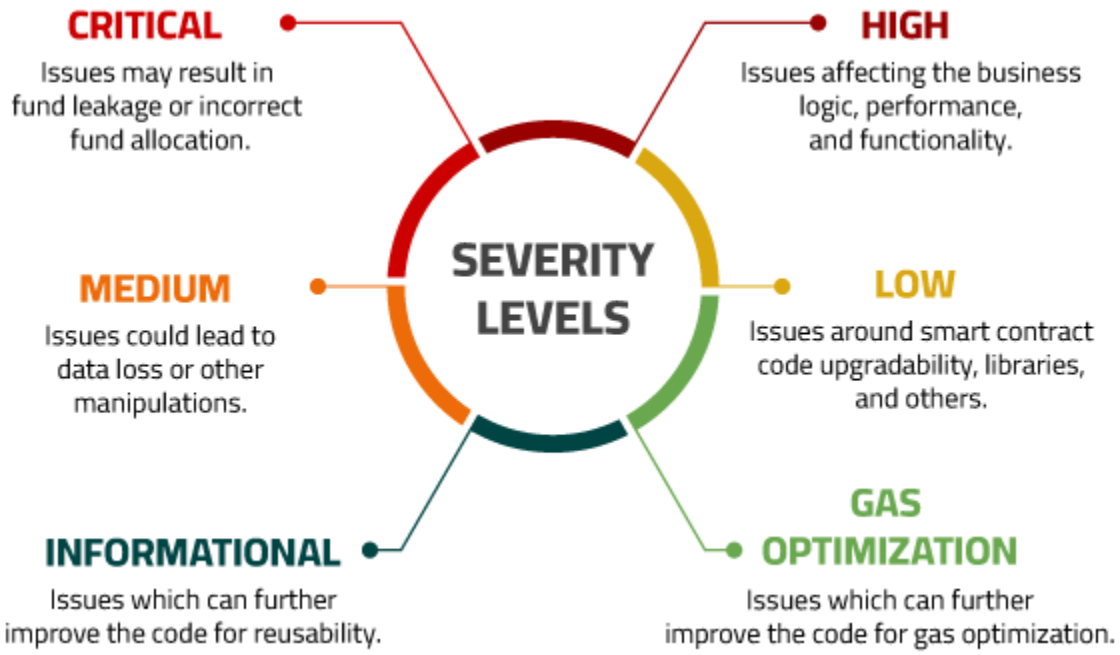
Audit Details

Project Name	Virtua
Platform	EVM
Contract	VANRYVestingContract.sol, VANRYVestingFactory.sol
Languages	Solidity
GitHub Link	https://github.com/TerraVirtuaCo/vanar-vesting-contract/tree/main
Commit	7fe2510dd0c53719b29ccae8f14f84923ede99af
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	1	2	2	2	-
Redacted	-	-	-	-	-	-
Acknowledged	-	1	-	1	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Findings	Risk	Status
1.	Centralizaion Related Risks	High	Acknowledged
2.	Future Consideration of Reentrancy	High	Fixed
3.	Missing Input Validation	Medium	Fixed
4.	SafeTransfer Methods Missing	Medium	Fixed
5.	Inherited Ownable uses a single-step ownership transfer	Low	Acknowledged
6.	State variable should be immutable	Low	Fixed
7.	Remove unnecessary imports	Low	Fixed
8.	Don't calculate keccak256 for ROLE_Admin	Informational	Fixed
9.	Recaculation of initialUnlockTokenAmount	Informational	Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

Issue	Centralizaion Related Risks
Description	In the contract, both VANRYVestingFactory and VANRYVestingContract grant authority to the roles onlyOwner and onlyAdmin over several functions. Any compromise of the owner/admin account may enable a hacker to exploit the contracts and withdraw Vested tokens from the vesting contracts.
Recommendation	The risk stems from the current project design, and potential improvements in security operations and decentralization may require iterative adjustments. These issues, in most cases, cannot be entirely resolved at the current stage. We advise the client to meticulously manage the private key of privileged accounts to mitigate potential hacking risks. In general, we strongly recommend enhancing centralized privileges or roles in the protocol through a decentralized mechanism or smart-contract-based accounts with advanced security practices, such as multisignature wallets. Here are some feasible suggestions that could also mitigate potential risks at different levels in the short term, long term, and permanently: Short Term: Timelock and Multi sign (⅔, ⅗) combination mitigate by delaying the sensitive operation and avoiding a single point of key management failure. • Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND • Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised; AND • A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience. Long Term: Timelock and DAO, the combination, mitigate by applying decentralization and transparency. • Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations; AND • Introduction of a DAO/governance/voting module to increase transparency and user involvement. AND

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

	• A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience. Permanent: Renouncing the ownership or removing the function can be considered fully resolved. • Renounce the ownership and never claim back the privileged roles. OR • Remove the risky functionality
Status	**Acknowledged**
Client Response	We have a new role to manage the vested tokens (ROLE_FUNDS_MANAGER) that can only withdraw. We can decide if this tole needs to be granted to an EOA or a multisig wallet. Also we are happy with the centralized control as all the owner of the factory contract do is to deploy new contract or change the status of a deployed contract at factory level. In case of any hack a new factory contract can be deployed in worst case.
Auditor Response	No follow-up is required.

Issue	Future Consideration of Reentrancy
Description	This finding is created in consideration of support for tokens with hooks or callback features where at the time of transfer of token in VANRYVesting.releaseTokens() the states are updated after the transfer.
Recommendation	We recommend keeping this information in consideration, and, if tokens with hooks are to ever be supported, we recommend following the check-effect-interaction pattern in order to prevent any potential reentrancy in this case.
Status	**Fixed**
Client Response	Considering only Vanry token will be used for ERC20 tokens then no scenario of reentrency will be raised.
Auditor Response	No follow-up is required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Medium severity issues

Issue	Missing Input Validation
Description	In function deployVestingContract input validation for parameters are missing.
Recommendation	We recommend to put the validations checks for all the input validations
Status	**Fixed**
Client Response	Input validation is already done at vesting contract level which is related to the vesting contract. All the validations which are related to factory contract are already in place. #Fixed contract
Auditor Response	No follow-up is required.

Issue	SafeTransfer Methods Missing
Description	It is good to add a require() statement that checks the return value of token transfers or to use something like a safeTransfer/safeTransferFrom unless one is sure the given token reverts in case of a failure. Failure to do so will cause silent failures of transfers and affect token accounting in the contract.
Recommendation	Replace transfer with safeTransfer as ERC20 token implementation can be any. If transferFrom() does not return a value (e.g., USDT), the transaction reverts because of a decoding error. Revert without error.
Status	**Fixed**
Client Response	Our vested token doesn't support safeTransfer function.
Auditor Response	No follow-up is required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

Issue	Inherited Ownable uses a single-step ownership transfer
Description	During the code review, It has been noticed that the contracts use single-step ownership transfer in the contract VANRYVestingFactory.
Recommendation	Consider using the Ownable2 contract version for the implementation.
Status	**Acknowledged**
Client Response	This is known and we can go with this. No action at our end
Auditor Response	No follow-up is required.

Issue	State variable should be immutable
Description	As VESTED_TOKEN and is a public constant and dependent on network and vestingEndTime is initilized once in constructor.
Recommendation	We recommend to make the VESTED_TOKEN, vestingEndTime as immutable and initilize at the time of VANRYVesting constructor contract.
Status	**Fixed**
Client Response	1) vestingEndTime is now marked as immutable. 2) We are keeping the address of the token as constant to save on gas. We also know that it is network dependent and will be updated prior to deployment with the token address on the respective network.
Auditor Response	No follow-up is required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Issue	Remove unnecessary imports
Description	As string is already imported in AccessControl.sol
Recommendation	We recommend to remove the below line from VANRYVESTING contract. `import "@openzeppelin/contracts/utils/Strings.sol";`
Status	**Fixed**
Client Response	NA
Auditor Response	No follow-up is required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

Issue	Don't calculate keccak256 for ROLE_Admin
Description	The keccak256 calculation should not be done onchain.
Recommendation	We recommend to directly use the bytes32 value of keccak256("ROLE_ADMIN") in the contract and mention a comment for the same.
Status	**Fixed**
Client Response	NA
Auditor Response	No follow-up is required.

Issue	Recaculation of initialUnlockTokenAmount
Description	On line 148 the calculation of (_initialUnlockPercentage * totalAmountInEther) is done unnecessary.
Recommendation	We recommend to use the initialUnlockTokenAmount instead. tokensPerSecond = (totalAmountInEther - initialUnlockTokenAmount / 100)) / _vestingDuration;
Status	**Fixed**
Client Response	NA
Auditor Response	No follow-up is required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Concluding Remarks

While conducting the audits of the Virtua Vesting Contract, it was observed that the contracts contain High, Medium, and Low severity issues. Our auditors suggest that the developers should resolve issues on priority before moving up with production. The recommendations given will improve the operations of the smart contract.

Note: The fixes have been done by the Virtua team liaising with ImmuneBytes security engineers.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Virtua platform or its product, nor is this audit investment advice.
Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES
AUDITS